Hyperparameter Optimization for Image Classification

A reusable methodology for picking the right network

Jacob Celniker | Anand Gangavarapu | Paolo Gonzalez

CSCI 611, California State University, Chico, Spring 2026

1. Overall Context and Relevant Work

Image classification has become a routine deep learning task, but selecting the right network for a given problem is anything but routine. The PyTorch and Hugging Face ecosystems make dozens of pretrained convolutional networks available with a single import, but engineers frequently default to whichever architecture they happen to know best, ResNet50 being the most common reflex choice. This default ignores the fact that real-world deployment is shaped at least as much by model size, inference latency, and memory footprint as it is by raw accuracy. For example, a 92% ResNet50 that cannot run in under 10 ms on the target device may be, in practice, less useful than an 89% MobileNet that can.

The work in this project sits at the intersection of three established threads. First, the architecture families themselves: ResNet (He et al., 2015) introduced residual connections that allowed networks to scale to dozens of layers without vanishing gradients; MobileNetV2 (Sandler et al., 2018) used depthwise separable convolutions and inverted residuals to dramatically reduce parameter count for mobile inference; and EfficientNet (Tan and Le, 2019) introduced compound scaling, jointly increasing depth, width, and resolution to achieve favorable cost/accuracy tradeoffs. Each represents a deliberate point in the design space.

Second, the practice of transfer learning, where features learned on ImageNet-1k are fine-tuned for downstream tasks. This is now the default for most practical computer vision work and shapes which hyperparameters matter and at what scale.

Third, automated hyperparameter optimization. We use Optuna (Akiba et al., 2019), which implements a Tree-structured Parzen Estimator (TPE) for Bayesian optimization over the search space. TPE is well suited to the small-to-medium scale we operate at because it learns from previous trials rather than re-exploring uniformly.

The target dataset is the Food Image Classification Dataset on Kaggle [1], containing roughly 24 thousand images across 35 labeled food categories (one class, kuzhi_paniyaram, contained a corrupted set and was removed, leaving 34 effective classes). Food is a useful testbed because it features both significant intra-class variation (a sandwich looks very different from one image to the next) and high inter-class similarity (taco and taquito share most visual features). It is also free of the licensing concerns that complicate medical or facial datasets.

To be clear about scope, the goal of this project is not to build the best food classifier. The goal is to build a reusable methodology for picking the right network on any classification problem, and to demonstrate that methodology on a non-trivial task.

2. High-Level Framework

The framework is designed for apples-to-apples comparison across architectures as fairly as possible. It is a trivial task to grab three networks, train them with different scripts, different epochs, different augmentation, and different validation splits, and then argue about which is best. We instead constrain everything that is not the architecture so that the comparison is meaningful.

2.1 What is held constant

Dataset split. A fixed train/val/test partition is generated once with seed 42, written to a YAML manifest (configs/data_split.yaml), and read by every run thereafter. Every model trains on the exact same images and is evaluated on the exact same held-out test set.

Training and evaluation pipeline. A single script handles model construction, training loop, checkpointing, and evaluation. There is no per-model code path that could quietly advantage one architecture.

Starting weights. All three architectures load torchvision ImageNet-1k pretrained weights. None is trained from scratch.

Search space for common hyperparameters. Learning rate, weight decay, dropout in the new classifier head, batch size, and optimizer choice are sampled from identical ranges for every architecture.

2.2 What varies

Each architecture is allowed one or two extra hyperparameters that target a known characteristic of that architecture:

ResNet50: label smoothing. ResNet is prone to overconfident logits at high capacity. Smoothing acts as a soft regularizer.

EfficientNet-B0: cosine LR schedule (on/off). EfficientNet was originally trained with a cosine schedule, and we wanted to see whether keeping it on or replacing it with a flat LR was preferable for fine-tuning.

MobileNetV2: freeze backbone (on/off). The lightest model carries the highest risk of overfitting at fine-tune time, so we allow the search to choose between fine-tuning the whole network and training only the new head.

2.3 The Three Architectures at a High Level

Before getting into specifics, it is worth pausing on what the candidate architectures look like. The three networks were chosen because they sit at deliberately different points in the parameter and design-complexity space. MobileNet is a lightweight mobile-first design, Efficient net is a parameter-efficient compound-scaled design, and ResNet is a relatively large and powerful residual design.

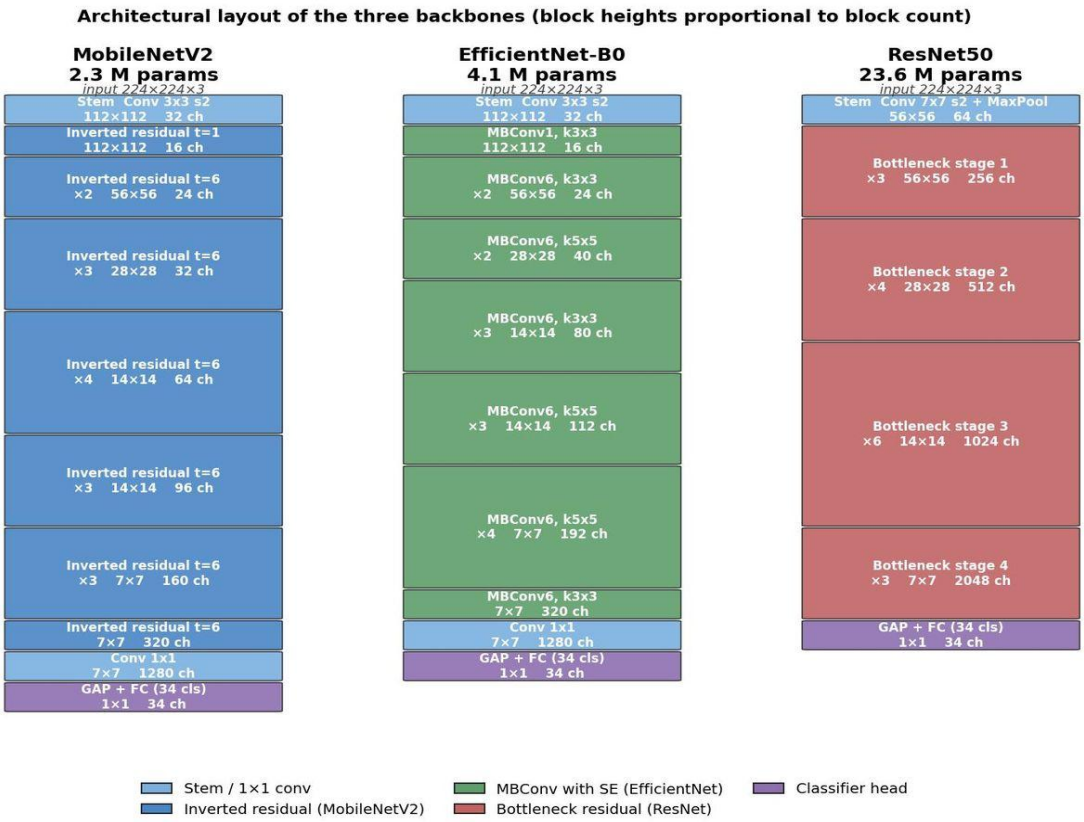

Figure 1. Block-level layout of the three backbones. Box height is proportional to the number of repeated blocks in that stage; channel counts and output spatial resolutions are annotated. The shared 1280-channel pre-classifier conv in MobileNetV2 and EfficientNet-B0 contrasts with ResNet50's 2048-channel final stage.

Table 1 summarizes the key architectural differences. Note that the parameter counts shown are backbone-only (the original 1000-class ImageNet FC layer has been replaced with a 34-class FC for our task).

	MobileNetV2	EfficientNet-B0	ResNet50
Year	2018	2019	2015
Reference	Sandler et al.	Tan and Le	He et al.
Backbone params	2.3 M	4.1 M	23.6 M
FLOPs at 224x224	~0.31 G	~0.39 G	~4.12 G
Building block	Inverted residual with linear bottleneck	MBConv with squeeze-and-excitation	Bottleneck residual (1x1, 3x3, 1x1)
Block count	17	16 (across 7 stages)	16 (across 4 stages)
Activation	ReLU6	Swish (SiLU)	ReLU
Final conv channels	1280	1280	2048
Key innovation	Mobile-first depthwise separable convs	Compound scaling of depth, width, and resolution	Skip connections enabling 50+ layers
ImageNet-1k top-1 (paper)	72.0%	77.3%	76.0%

Table 1. Architectural comparison of the three backbones evaluated in this work.

The three architectures span roughly an order of magnitude in parameter count (2.3 M to 23.6 M) and FLOPs (~0.31 G to ~4.12 G), and they use three structurally different building blocks. This was done intentionally so that any winning configuration we identify reflects a genuine architectural property rather than tuning noise within a single family.

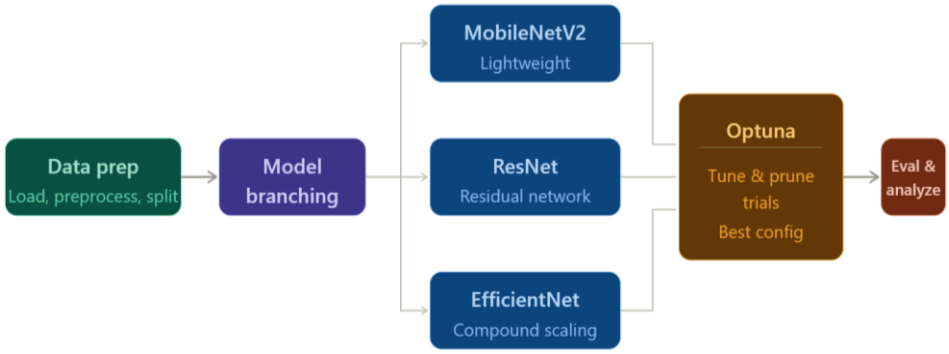

Figure 2. High-level pipeline. Data prep produces a fixed split manifest; model branching routes to one of three architectures; Optuna tunes and prunes trials; the eval/analyze step produces the final report and plots.

3. Implementation Details

3.1 Dataset and Split

After filtering one corrupted class, the working dataset contains 23,873 images across 34 classes. The class distribution is moderately imbalanced. The most frequent class (Hot Dog, Baked Potato, Crispy Chicken, all roughly tied) accounts for 6.3 percent of images each, while the least frequent class (paani_puri) accounts for 0.8 percent. We use a stratified 80/10/10 split, yielding 19,092 training images, 2,377 validation images, and 2,404 test images. The split manifest is generated by scripts/generate_split_manifest.py and stored as YAML so that every downstream run reads the same partition.

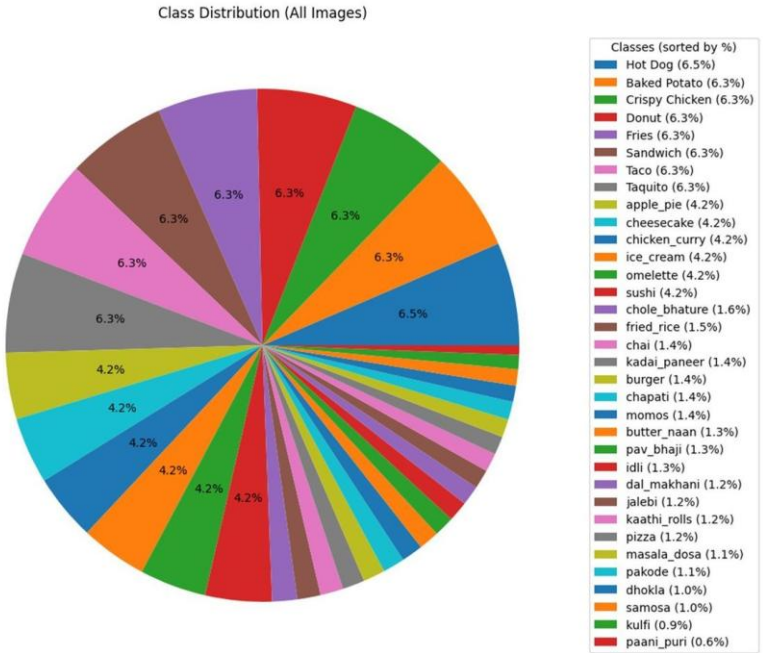

Figure 3. Class distribution across the 34 categories. The most frequent class (Hot Dog) accounts for 6.5 percent of images; the least frequent (paani_puri) accounts for 0.6 percent.

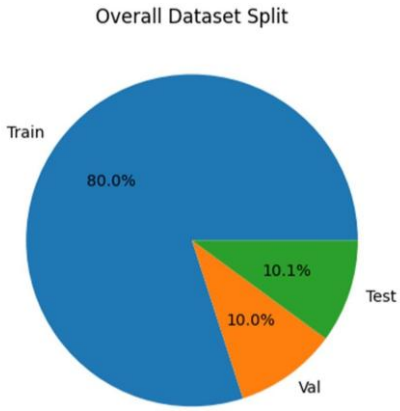

Figure 4. Overall train/val/test split (80/10/10), yielding 19,092 training images, 2,377 validation images, and 2,404 test images.

3.2 Search Space

The common search space, sampled identically for every architecture, is defined in src/search_space.py as the CommonSearchSpace dataclass:

```
class CommonSearchSpace:
    """The hyperparameters every architecture searches over.

    These mirror the standard tuning levers in image classification:
      * learning rate (log scale, by far the most important)
      * weight decay (log scale, regularization)
      * optimizer choice (SGD-momentum vs AdamW)
      * batch size (categorical, hardware-bounded)
      * dropout in the new classifier head
    """
    lr_min: float = 1e-5
    lr_max: float = 1e-2
    weight_decay_min: float = 1e-6
    weight_decay_max: float = 1e-3
    dropout_min: float = 0.0
    dropout_max: float = 0.5
    batch_sizes: tuple = (16, 32, 64)
    optimizer_choices: tuple = ("adamw", "sgd")
```

Each architecture additionally exposes a small extra_space_fn that adds its specific knobs (label_smoothing for ResNet, cosine_lr_schedule for EfficientNet, freeze_backbone for MobileNet).

3.3 Objective Function

The Optuna objective is built by a factory in src/objective.py so the same code services all three architectures. Each trial executes the following steps:

Step 1: sample a hyperparameter configuration.

```
def objective(trial: optuna.Trial) -> float:
    # 1. Sample a hyperparameter configuration.
    common_hp = sample_common_hparams(trial, common_space)
    extra_hp = extra_space_fn(trial) if extra_space_fn is not None else {}
    hparams = {**common_hp, **extra_hp}
```

Step 2: build the data pipeline. Because batch_size is a sampled hyperparameter, the dataloaders are rebuilt inside the objective rather than outside it. This is slightly wasteful but it is the only way to honor batch size as a tuning knob.

```
    # 2. Build the data pipeline.
    manifest, train_loader, val_loader, _ = get_dataloaders(
        manifest_path=manifest_path,
        batch_size=hparams["batch_size"],
        num_workers=num_workers,
    )
```

Step 3: build the model and apply any per-architecture extras. For MobileNet this is where freezing happens, before the optimizer sees the parameter list, so frozen parameters are excluded from optimization.

```
    # 3. Apply optional per-arch extras BEFORE building the optimizer
    #    so frozen params are excluded from optimization.
    _freeze_backbone_if_requested(built.model, hparams)

    # 4. Build the optimizer using sampled lr/optimizer/weight_decay.
    optimizer = build_optimizer(
        built.model,
        name=hparams["optimizer"],
        lr=hparams["lr"],
        weight_decay=hparams["weight_decay"],
    )
    criterion = _build_criterion(hparams)  # honors label_smoothing if sampled
    scheduler = _build_scheduler(optimizer, hparams, epochs=epochs_per_trial)
```

Step 4: run training. The training loop in src/train.py handles epoch loop, validation, checkpointing, and intermediate reporting back to Optuna for pruning. The objective returns the best validation accuracy seen across epochs.

```
    # 5. Run training. If the trial gets pruned, run_training raises
    #    optuna.TrialPruned, which Optuna catches and records.
    result: TrainingResult = run_training(
        model=built.model,
        train_loader=train_loader,
        val_loader=val_loader,
        epochs=epochs_per_trial,
        optimizer=optimizer,
        criterion=criterion,
        scheduler=scheduler,
        checkpoint_dir=per_trial_dir,
        optuna_trial=trial,
    )
    return result.best_val_acc
```

3.4 Study Execution

Each architecture runs as a separate Optuna study with its own SQLite database under outputs/optuna_studies, so runs are resumable. The make_objective factory function creates a callable objective function for each of the architectures, with resnet shown below.

```
objective = make_objective(
    arch="resnet50",
    manifest_path=args.manifest,
    num_workers=args.num_workers,
    epochs_per_trial=args.epochs,
```

```
    checkpoint_dir=output_dir / "checkpoints" / "resnet50",
    common_space=CommonSearchSpace(),
    extra_space_fn=resnet_extra_space,
)
study.optimize(objective, n_trials=args.trials, gc_after_trial=True)
```

Each trial uses 8 epochs by default, with early pruning via Optuna's MedianPruner. After the study ends, the best trial's checkpoint is re-evaluated on the test split for the final reported numbers.

3.5 Analysis and Visualization

Three additional scripts consume the trained checkpoints to produce the analysis figures. scripts/analyze_confusions.py builds per-architecture confusion matrices (row-normalized) and per-class accuracy bars.

scripts/benchmark_speed.py measures throughput at batch size 32 and single-image latency at batch size 1, with a warmup pass to exclude CUDA initialization overhead.

scripts/gradcam_compare.py renders Grad-CAM heatmaps for selected test images across all three models, with the top-3 predictions overlaid.

4. Test Results and Analysis

4.1 Optuna Study Summary

Each architecture was studied independently. Table 2 reports the best configuration found by its Optuna study, alongside trial counts and validation accuracy.

	MobileNetV2	EfficientNet-B0	ResNet50
Trials run	31	25	20
Best trial #	29	5	18
Best val acc	90.41%	92.22%	92.26%
Learning rate	2.17e-3	8.11e-3	4.26e-5
Weight decay	6.41e-4	2.12e-4	1.96e-5
Dropout	0.197	0.470	0.133
Batch size	32	64	16
Optimizer	SGD	SGD	AdamW
Per-arch extra	freeze_backbone = false	cosine_lr_schedule = false	label_smoothing = 0.098

Table 2. Best hyperparameter configuration per architecture, as selected by Optuna.

Three observations stand out. First, the best optimizer differs between architectures: ResNet50 prefers AdamW at a very low learning rate (4.26e-5), while the other two prefer SGD with momentum at much higher rates (around 2e-3 to 8e-3). This may be consistent with common practice but it is not obvious from theory alone; Optuna found it empirically. Second, the best batch sizes are also different, with 16 for ResNet50, 32 for MobileNetV2, 64 for EfficientNet-B0. Third, the convergence is fast. ResNet50 hit its best at trial 18, EfficientNet hit its best at trial 5, MobileNet at trial 29. Optuna's TPE allocated more trials where the search space had more uncertainty.

4.2 Hyperparameter Importance

Optuna also reports a hyperparameter importance score per study, computed via fANOVA over the completed trials. This metric quantifies the net absolute impact of each tunable hyperparameter. The result is surprisingly that each architecture has a completely different dominant hyperparameter.

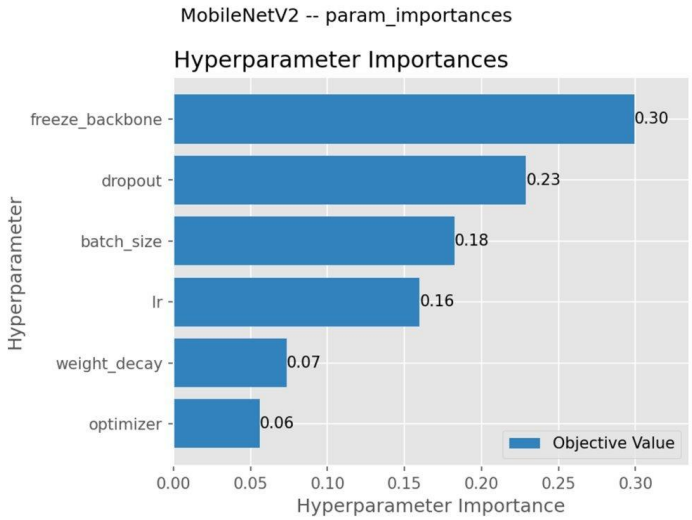

Figure 5. MobileNetV2 hyperparameter importance. The single most impactful choice is whether to freeze the backbone.

For MobileNetV2, the freeze_backbone toggle accounts for roughly 30 percent of the explained variance, with the best answer being do not freeze (shown by Table 2). This is the cleanest signal in the entire study: the lightweight architecture has enough capacity to fine-tune end-to-end on 19k images without overfitting, and locking the backbone wastes the available training signal.

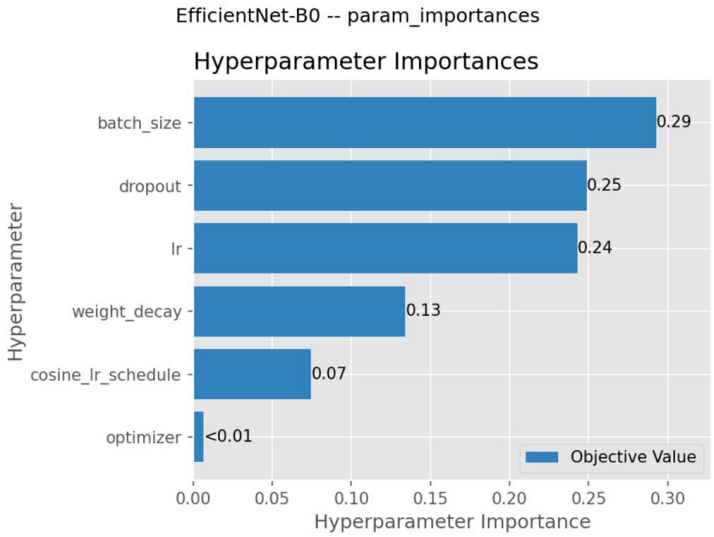

Figure 6. EfficientNet-B0 hyperparameter importance. Batch size, dropout, and learning rate dominate.

For EfficientNet-B0, no single hyperparameter dominates; batch size (0.29), dropout (0.25), and learning rate (0.24) all matter at similar magnitudes. Notably, the cosine LR schedule contributes only 0.07 importance and the best configuration disables it. We interpret this as evidence that for a short fine-tuning regime, the implicit early-epoch warmup that the original EfficientNet training schedule provided is no longer needed.

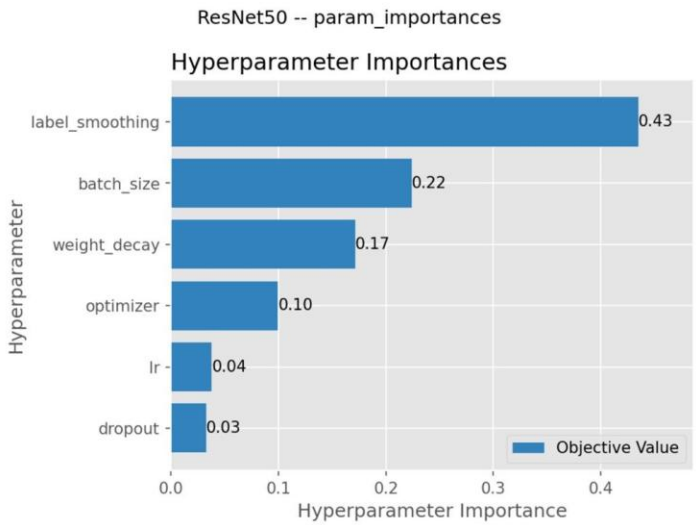

Figure 7. ResNet50 hyperparameter importance. Label smoothing alone accounts for 43 percent of the explained variance.

For ResNet50, label smoothing dominates with an importance of 0.43, far above batch size at 0.22. The best configuration uses a small but nonzero smoothing of approximately 0.098. This matches the architecture's known tendency to produce overconfident logits at scale. Without label smoothing, ResNet50 would have looked considerably worse in the study.

4.3 Final Test Set Results

Each architecture's best checkpoint was evaluated once on the held-out test set. Results in Table 3.

Architecture	Params	Best val	Test top-1	Test top-5	ImNet ref
MobileNetV2	2.3 M	90.41%	91.06%	99.25%	~72%
EfficientNet-B0	4.1 M	92.22%	92.22%	99.46%	~77%
ResNet50	23.6 M	92.26%	92.72%	99.29%	~76%

Table 3. Test set performance per architecture. ImageNet reference column is torchvision's reported top-1 on ImageNet-1k for context.

The headline result is that EfficientNet-B0 matches ResNet50 within 0.5 percentage points of top-1 accuracy while using less than one-fifth of the trainable parameters. The visual cost/accuracy tradeoff is shown in Figure 8.

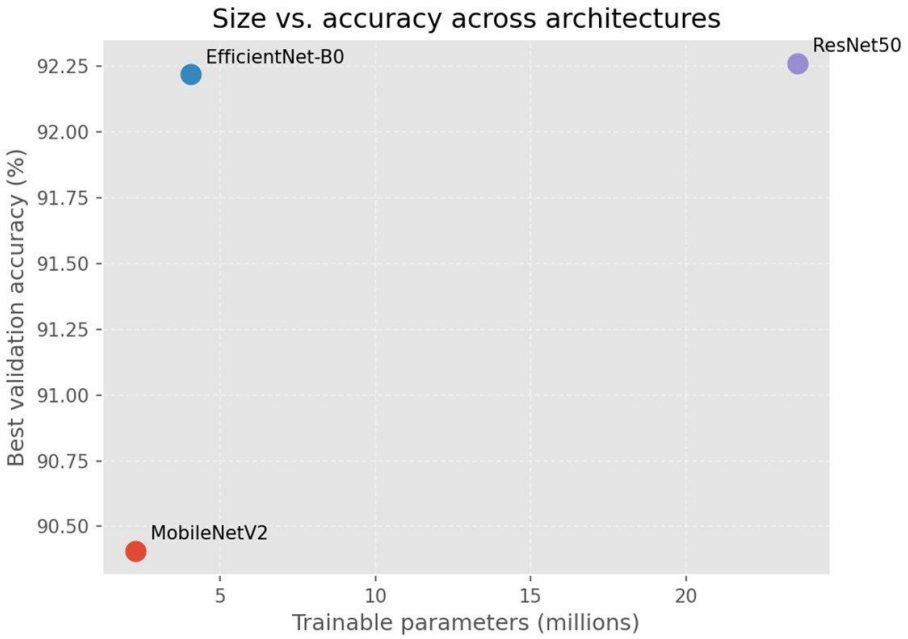

Figure 8. Size versus accuracy across architectures. EfficientNet-B0 is the cost/performance sweet spot; ResNet50 spends 5x the parameters for 0.5 percent more test top-1.

Comparison with the reference baselines (the torchvision ImageNet-1k numbers in Table 3) is informative for a different reason. On ImageNet, ResNet50 actually performs slightly worse than EfficientNet-B0 (76% vs 77%), and MobileNetV2 trails both (72%). On our 34-class food task, all three models exceed 90 percent. Two factors explain the gap. First, our task is much narrower (34 classes vs 1000). Second, transfer learning carries most of the load: the backbones have already learned the visual primitives, and the fine-tune step just rearranges those features into food categories.

4.4 Per-Class and Confusion Analysis

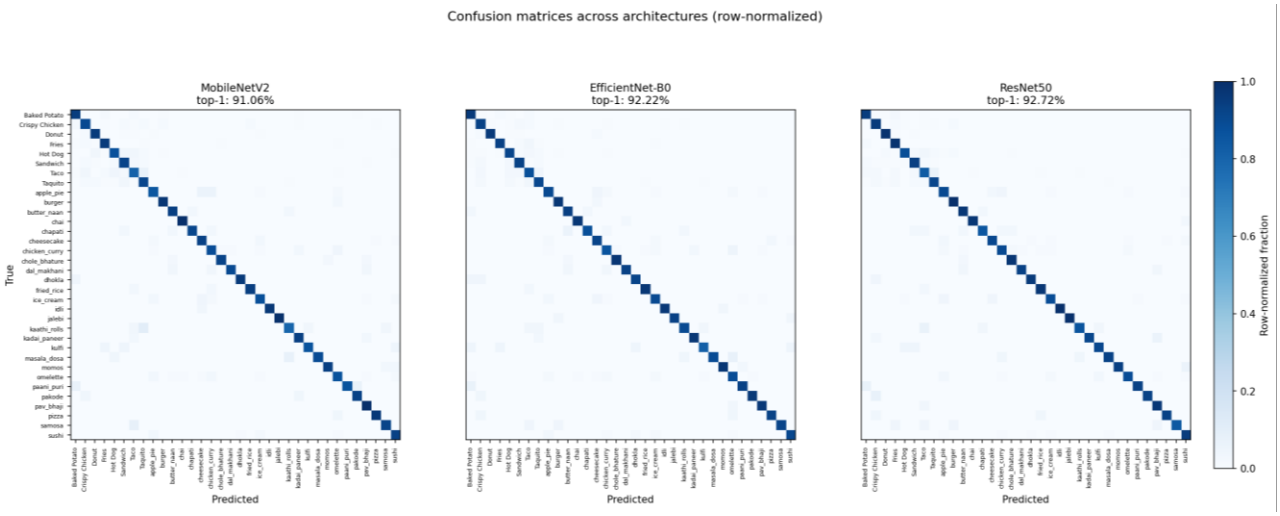

Figure 9. Row-normalized confusion matrices for all three architectures on the test set. All three show the same off-diagonal pattern.

The confusion matrices in Figure 9 tell a consistent story across architectures. The largest off-diagonal entries are not random; they are systematic. Taco gets confused with Taquito most frequently across all three models. apple_pie and cheesecake confuse in both directions. The fact that the same errors appear regardless of architecture, regardless of trained-from-scratch vs fine-tuned, regardless of model size, is strong evidence that the residual error is dataset difficulty rather than model limitation.

This is corroborated by top-5 accuracy, which is saturated at over 99 percent for all three models. The correct class is almost always in the model's top 5; the residual error is fine-grained discrimination between visually similar classes.

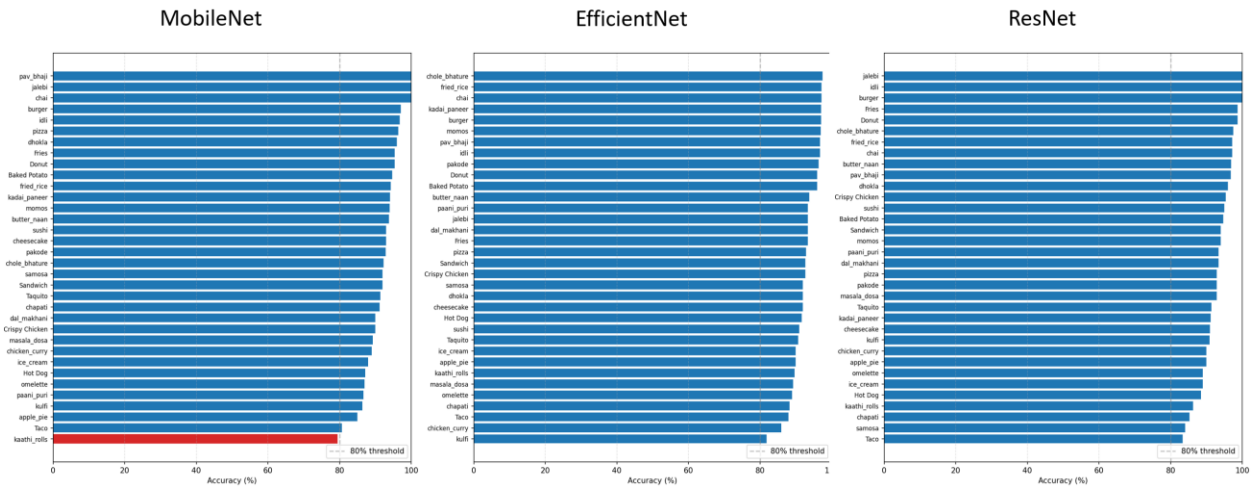

Figure 10. Per-class accuracy for each architecture, sorted descending within each panel. The kaathi_rolls class under MobileNetV2 (highlighted in red) is the only class to fall below the 80 percent threshold across all three models.

Per-class accuracy (Figure 10) shows that almost all classes clear 80 percent on all three models. The exception is the kaathi_rolls class under MobileNetV2, which falls just below 80 percent and is highlighted in red. The hardest classes are consistent across models being Taco, kulfi, samosa, and chapati which appear near the bottom of every chart. The easiest classes (jalebi, idli, burger, fries) appear near the top. Again, the agreement across architectures supports the interpretation that these errors are about the data, not the models.

4.5 Inference Speed

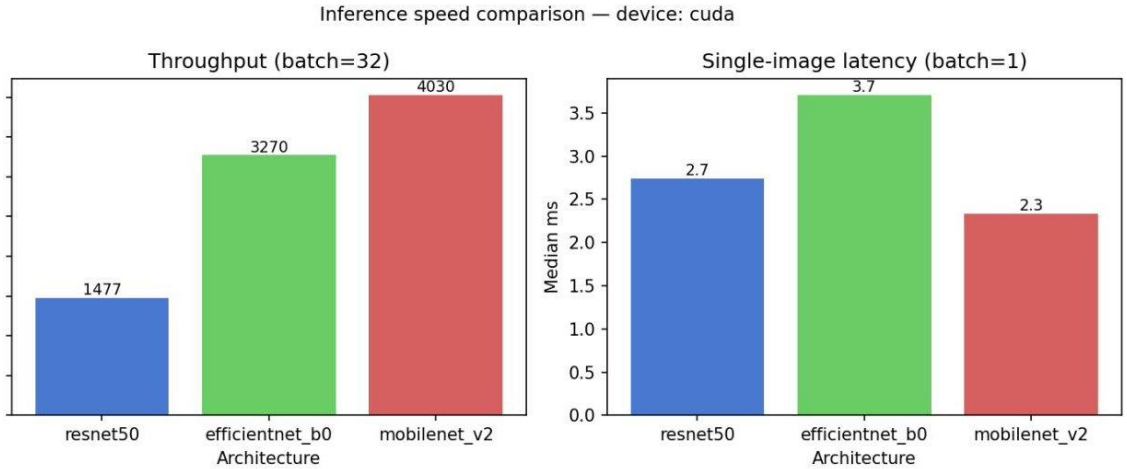

Figure 11. Inference speed on CUDA. Throughput is measured at batch=32; latency is measured at batch=1.

The speed numbers complicate the story slightly. At batch=32 throughput, the ordering is as expected: MobileNetV2 fastest (4,030 images/sec), EfficientNet-B0 in the middle (3,270 images/sec), ResNet50 slowest (1,477 images/sec). MobileNetV2 is 2.7x faster than ResNet50 at batch throughput.

At batch=1 single-image latency, however, the ordering breaks. MobileNetV2 is fastest at 2.3 ms, but ResNet50 is second at 2.7 ms, and EfficientNet-B0 is slowest at 3.7 ms. This is a real and useful finding. A possible explanation is that EfficientNet's depthwise separable convolutions, which are highly parameter-efficient and very fast at large batch, do not amortize well at batch=1 on the GPU because each depthwise op launches a small kernel. ResNet50's plain 3x3 convs, despite being more expensive in FLOPs, fit the GPU's preferred memory access pattern at single-image inference. The implication is that for latency-critical single-image inference (for example, a mobile camera app that needs to classify one frame at a time), the architecture that wins the throughput benchmark is not necessarily the one to deploy.

4.6 Grad-CAM Interpretation

Grad-CAM visualizations let us inspect what regions each architecture attends to when predicting a class. We compare two depths: an early convolutional stage (roughly the first quarter of the backbone) and the final stage. The expectation from the CNN literature is that early features fire on textures, edges, and local primitives, while late features integrate them into object-level semantics. We picked three test images that exercise the methodology in different ways: a clean single-object scene, a multi-object scene, and a borderline class that produces disagreement between architectures.

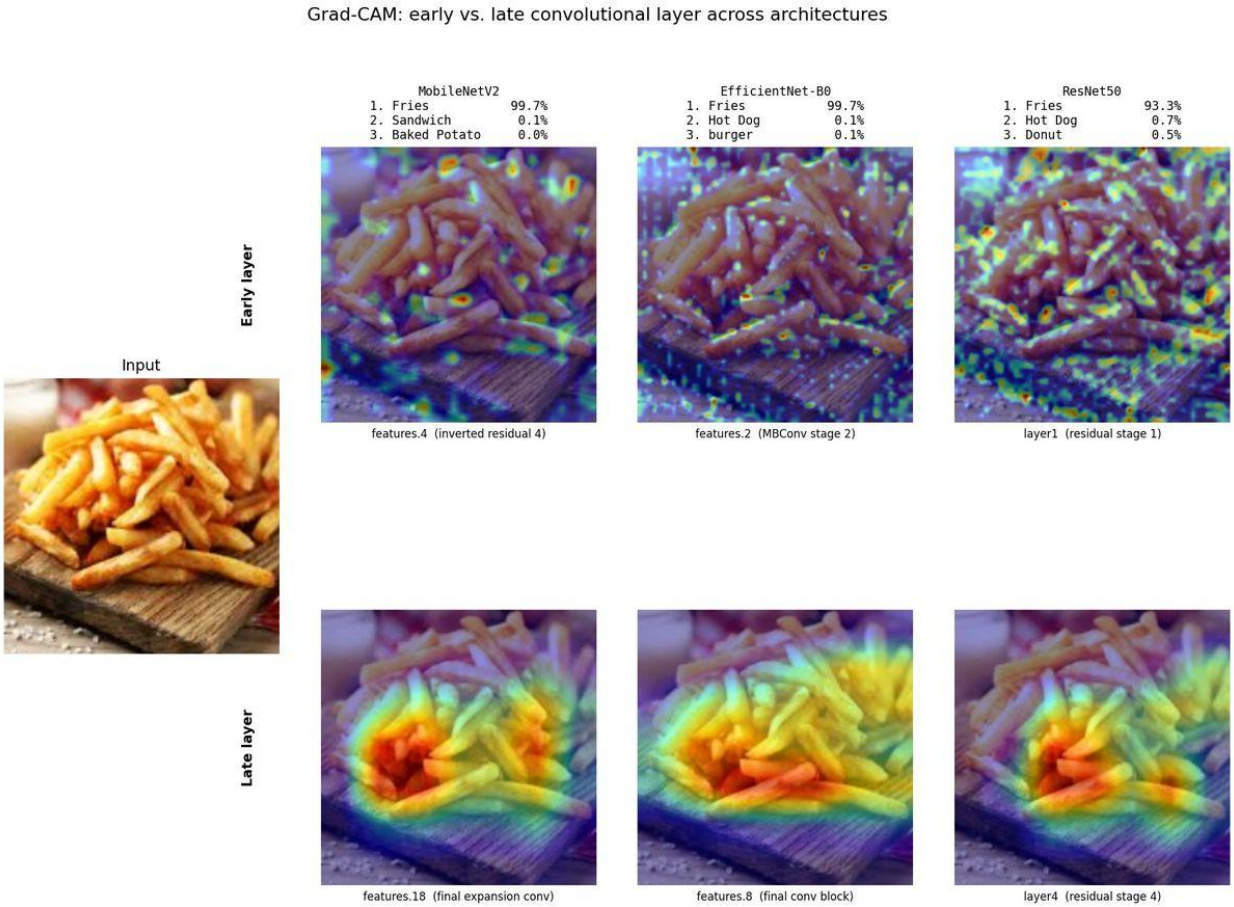

Figure 12. Grad-CAM comparison on a Fries image. Top row: early convolutional stage. Bottom row: final stage. Top-3 predictions are printed above each column.

Figure 12 is the easy case. All three architectures correctly call this Fries with very high confidence (MobileNetV2 and EfficientNet-B0 at 99.7 percent, ResNet50 at 93.3 percent). The early-layer heatmaps are scattered and high-frequency, lighting up edges of individual fries and the texture of the cutting board behind them. The late-layer heatmaps look very different: a single concentrated red blob over the center of the pile, with the wood plank and background discounted. This is the textbook story of CNN hierarchy, except now shown across three different architectures simultaneously. The pattern is consistent regardless of model family.

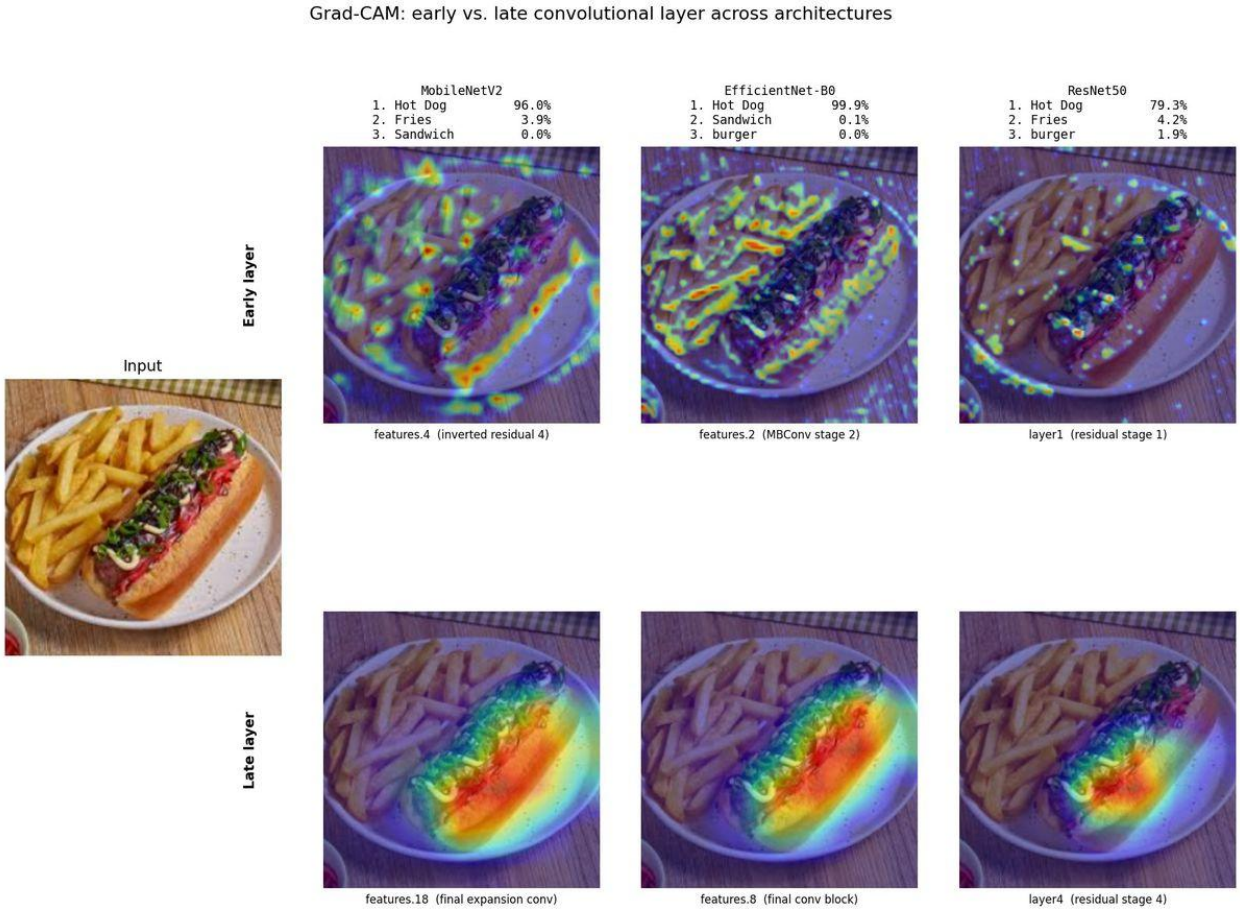

Figure 13. Grad-CAM on an image containing a hot dog and a side of fries. Early layers respond to both foods; late layers commit to the hot dog and suppress the fries.

Figure 13 reveals what happens when the early features themselves do not yet pick a single object. The early-layer heatmaps for all three architectures fire on the hot dog and on the fries, picking up the edges of both. The decision about which one is the answer happens later. By the final convolutional stage, all three models have placed the red zone squarely on the hot dog bun and the fries are suppressed to near-zero attention, despite occupying roughly the same image area. The top-1 confidences reflect how aggressively each model commits with EfficientNet-B0 at 99.9 percent, MobileNetV2 at 96.0 percent, ResNet50 at 79.3 percent (with a small 4.2 percent leak to Fries, the next-best class). This shows what happens when an image classification network is given an image with two labelled classes simultaneously. The network is forced to produce one label, and the level of certainty depends on how aggressively the late layers suppress the alternative.

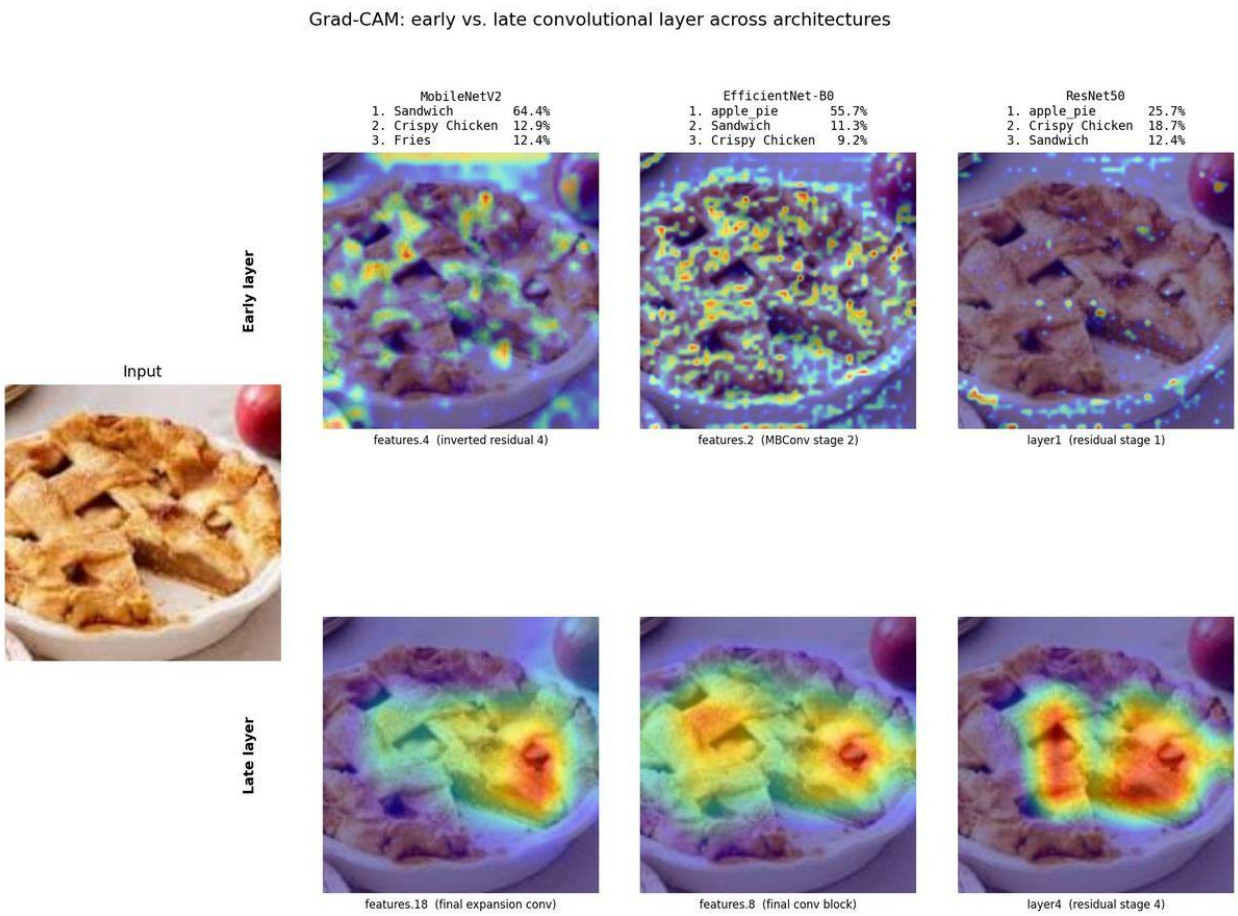

Figure 14. Grad-CAM on an apple_pie image. MobileNetV2 misclassifies as Sandwich; EfficientNet-B0 and ResNet50 correctly identify apple_pie but with relatively low confidence.

Figure 14 is the most informative panel. The image is clearly an apple pie with a lattice crust, but the model verdicts diverge. MobileNetV2 confidently picks the wrong class (Sandwich at 64.4 percent), and apple_pie does not even appear in its top three. EfficientNet-B0 and ResNet50 both pick apple_pie correctly but with much lower confidence than usual (55.7 and 25.7 percent respectively), with Sandwich and Crispy Chicken sitting close behind. The early-layer heatmaps fire on the lattice texture, which is the source of the problem, because woven crust patterns are visually similar to the cross-hatched grill marks on sandwich bread and the breading on crispy chicken. Notably, the late-layer heatmaps are broadly similar across the three architectures: all three concentrate on the central lattice region. The earlier observation that apple_pie confuses with cheesecake in the confusion matrix (Section 4.4) is the same phenomenon. Lattice and crumb topology are easy to mistake when the only signal you have is shape and texture.

4.7 What Worked

- Optuna with TPE. 20 to 31 trials per architecture were enough to find configurations that generalized to test. Trial pruning meant the cost was roughly half of what a naive run-every-trial-to-completion would have been.
- Transfer learning. Without it, none of these models would have cleared 80 percent on the test set in the time and compute we had. The pretrained ImageNet features are doing most of the work.
- Per-architecture extras. Label smoothing was decisive for ResNet50 (43 percent importance). Allowing each architecture its own one or two extra knobs cost us very little search budget and produced clearly visible effects.
- Modular pipeline. Building Optuna, training, and analysis as separate scripts that share a manifest and checkpoint format made the final report a one-command operation. This is the property that makes the methodology reusable on the next dataset.
- Fixed split via YAML manifest. Every comparison number in this report is over the same 2,404 test images. There is no possibility of accidental train/test leakage between runs.

4.8 What Did Not Work or Surprised Us

- Freezing the MobileNet backbone underperformed. We expected freezing to be at least competitive for the smallest model, given its overfitting risk, but the search consistently preferred end-to-end fine-tuning. The freeze_backbone hyperparameter had high importance precisely because the freeze=true trials were systematically worse.
- Cosine LR scheduling did not help EfficientNet-B0. Importance was only 0.07 and the best config disabled it. We suspect this is specific to the short fine-tune regime; cosine schedules earn their keep over longer training runs.
- Optimizer choice was nearly irrelevant for EfficientNet-B0 (importance under 0.01). For that model, the learning rate matters and the optimizer family essentially does not, within the searched range.
- EfficientNet-B0 was slower than ResNet50 at single-image latency despite having one-fifth the parameters. As discussed in Section 4.5, this is the depthwise-conv kernel launch overhead at batch=1. Parameter count alone is not a reliable proxy for latency.
- Trials per study could have been higher. With only 20 trials for ResNet50, the variance on importance estimates is not negligible. A larger budget (100+ trials) would tighten these numbers, and we list this as future work.
- The dataset has class imbalance and one corrupted class, which we had to remove. We caught this during initial pipeline development, but it would have been easy to miss had we trusted the published class count.

5. Conclusion and Future Work

5.1 Conclusion

We built a reusable hyperparameter-optimization pipeline that compares three CNN families head-to-head under controlled conditions. Applied to a 34-class food classification problem, the methodology produced three clear practical recommendations, depending on the mode of deployment.

For maximum accuracy without resource constraints, ResNet50 wins at 92.72 percent test top-1, at the cost of 23.6 M parameters. For the best cost/accuracy tradeoff, EfficientNet-B0 is the right choice at 92.22 percent test top-1 with only 4.1 M parameters. This was the most-cited finding from our analysis:

nearly ResNet50 accuracy at roughly one-fifth the parameter count. For latency-critical or edge deployment, MobileNetV2 at 91.06 percent and 2.3 ms single-image inference is the deployable answer.

More importantly, the value of the project is not in the food classifier. The same pipeline, with a different data manifest, would produce the same kind of analysis on any classification task. The same six common hyperparameters, the same per-architecture extras, the same TPE-driven search, and the same one-command report would all transfer unchanged.

5.2 Future Work

This project acts as a framework which could be built upon in many ways going forward. Here are some possibilities for how the work could be expanded.

Larger model variants. We tested only the smallest member of each family. Adding ResNet101, EfficientNet-B3 or B4, and MobileNetV3 would round out the comparison and would clarify whether the EfficientNet sweet spot extends to larger scales.

Augmentation in the search space. We held augmentation fixed (resize, crop, horizontal flip, normalize). RandAugment or AutoAugment with strength as a tunable hyperparameter could meaningfully change the winners.

Multi-objective optimization. Optuna supports multi-objective studies. Optimizing for accuracy and inference latency jointly would allow for practical, deployment-specific stipulations for what is preffered rather than ranking on accuracy alone.

Cross-dataset validation. The strongest test of methodology reusability is applying it to a substantially different domain (medical imaging, satellite, document understanding) and asking whether the same search space and per-architecture extras still produce useful answers.

References

[1] Food Image Classification Dataset, Harish Kumar (Kaggle). https://www.kaggle.com/datasets/harishkumardatalab/food-image-classification-dataset

[2] He, K., Zhang, X., Ren, S., and Sun, J. Deep Residual Learning for Image Recognition. CVPR 2016.

[3] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L.-C. MobileNetV2: Inverted Residuals and Linear Bottlenecks. CVPR 2018.

[4] Tan, M. and Le, Q. V. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. ICML 2019.

[5] Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. Optuna: A Next-generation Hyperparameter Optimization Framework. KDD 2019.

[6] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., and Batra, D. Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. ICCV 2017.

[7] Gonzalez, P. CSCI611_Group4. GitHub repository. https://github.com/paologonzalez/CSCI611_Group4